InferCell: inference-first whole-cell modelling

Tom Kimpson [1] **[Co-authors]** [1]

Abstract

Whole-cell models simulate a living cell from its molecular parts. They fix every parameter to one value, so each prediction is a single sample with no error bar. The models cannot say how much of a prediction the data support. Inference is hard because the models mix deterministic and stochastic formalisms, the stochastic paths are latent, and no parameter is shared across the boundary between the two blocks. We present INFERCELL, a Julia framework that makes inference a first-class operation in whole-cell modelling. Every parameter is a prior distribution with a recorded source. Modules couple through typed edges, and the framework checks the composition and lists every reduction from the published model. The framework can run the same gene-expression interface as an exact stochastic process or as a mean-field ODE, so the whole cell compiles to one differentiable ODE for gradient-based inference, at a declared and measured cost. We test the framework on a reduced model of the minimal cell JCVI-syn3A, with four ODE modules and three stochastic modules. An exact sampler passes simulation-based calibration on a two-gene reference system over 50 replicate datasets. [Approximate method result (B7): agreement with the exact posterior on M0, cost ratio.] [Full-scale result (B8): calibration on the reduced syn3A model; metabolite data shrink a gene-expression parameter they do not measure.] [Uncertainty-budget sentence: x% of the uncertainty in a prediction comes from parameters that no measurement supports.]

[Author summary (PLOS CB, $\leq$ 200 words) or Highlights and eTOC blurb (Cell Systems): write once the venue is chosen.]

[1][Affiliations]. Correspondence to: Tom Kimpson <[email]>.

Preprint. October 23, 2026.

1. Introduction

A whole-cell model simulates a cell from its molecular parts. The first such model described *Mycoplasma genitalium* with 28 coupled submodels (Karr et al., 2012). Later models covered *Escherichia coli* (Macklin et al., 2020) and the minimal cell JCVI-syn3A, first well stirred (Thornburg et al., 2022) and then in space and time over a full cell cycle (Thornburg et al., 2026). Each one joins metabolism, gene expression, growth and division in one simulation, and each reproduces measured behaviour of a real cell.

All of these models are forward simulators. Each one takes hundreds or thousands of kinetic parameters from the literature, sets each to one value, and integrates forward in time. The output is one trajectory. That trajectory is one sample from the distribution of trajectories that the parameter knowledge allows. The model does not know the width of that distribution. When a model predicts a doubling time, a reader cannot tell whether the prediction holds for every plausible parameter set or depends on a few uncertain values.

The missing error bar is a gap in the scientific reasoning. Every parameter in a whole-cell model is uncertain. A rate measured in one organism, at one temperature, in one medium, is then used in another organism under other conditions. The uncertainty is real, and it propagates through coupled nonlinear dynamics into every prediction. A framework that stores each parameter as a single number has no place to put this uncertainty. It cannot ask how uncertain a prediction is, and it cannot ask which data would reduce that uncertainty.

1.1. Why whole-cell models do not do inference

Bayesian inference answers both questions. It treats each parameter as a distribution, updates the distribution with data, and propagates the result into predictions. Systems biology argued for this approach long ago (Babtie & Stumpf, 2017). Whole-cell models still do not use it, for the following reasons.

1. *No place for uncertainty.* The frameworks store parameters as floating-point values in a dictionary. The source of each value is in a spreadsheet or a supplement, not in the model. When two sources disagree,

the code picks one without a record.

2. *Mixed formalisms.* Metabolism is a set of ordinary differential equations (ODEs). Gene expression is a stochastic jump process. The two exchange state on a fixed clock. No single likelihood and no standard sampler covers both.

3. *Latent paths.* The stochastic block has a hidden path: the exact times of each transcription and translation event. The likelihood of the data integrates over all such paths, and that integral has no closed form.

4. *Cost.* Inference needs many simulations. For a whole-cell model the number of simulations, not the cost of one, sets the budget.

5. *No shared parameters.* In the published JCVI-syn3A model, no rate constant appears in both blocks. The blocks couple only through state. The standard methods for modular inference pass information through shared parameters (Plummer, 2015; Jacob et al., 2017; Carmona & Nicholls, 2020). Here they have nothing to pass.

Table 1 compares current whole-cell frameworks on these points. Thornburg and coworkers balanced the metabolic parameters of JCVI-syn3A against each other with a Bayesian method (Thornburg et al., 2022). That step used literature values for metabolism only. It did not infer parameters from dynamic data through the coupled model.

1.2. This work

We present INFERCELL, a framework that puts inference first. We do not claim that full Bayesian inference over every parameter of a whole-cell model is possible today. We claim that the framework must support inference as a basic operation, so that a modeller can apply it where the data and the budget allow. Our contributions are:

1. **A design for inference-first composition** (Section 2). Every parameter is a prior with a recorded source. Modules couple through seven kinds of typed edge. The framework checks that a composition is complete before it runs, and it lists every departure from the published model.

2. **One interface with two descriptions** (Section 2). The same gene-expression modules run as an exact stochastic process or as mean-field ODEs. In the second form the whole cell compiles to one ODE problem that automatic differentiation can pass through.

3. **A reduced model of JCVI-syn3A with a real boundary** (Section 3). It keeps 22 metabolic reactions and 17 genes from the published model, and it keeps the two-way coupling between them.

4. **A map of how information crosses the boundary** (Section 4), and calibrated inference across it (Section 5). An exact sampler gives a calibrated reference. [An approximate method agrees with it at a fraction of the cost, and scales to the full reduced model.]

[Last paragraph: a one-sentence statement of the main result, written after B7 and B8.]

2. InferCell: an inference-first design

INFERCELL is written in Julia (Bezanson et al., 2017). It uses the SciML solvers for ODEs and jump processes (Rackauckas & Nie, 2017). This section gives the design ideas. The documentation gives the full interface (Section 6.2). Figure 1 shows the parts and how they connect.

2.1. Parameters are distributions with a source

Each parameter in INFERCELL is an object that holds a value, a prior distribution, a role and a record of its source. The record names the file, the table and the identifier the value came from, and the values that other files give for the same identifier. Each parameter also has one of four labels for how well its prior is informed:

- *balanced*: the value and width come from a balanced parameter table;
- *prior default*: the source gives no width, so the prior is wide;
- *asserted*: the source gives no uncertainty, so we set the prior;
- *not imported*: the parameter is not in the source.

The loader does not choose silently between two files that disagree. It stops and asks for a decision, and the decision is then part of the record. A user can ask the composed model which priors no measurement supports. That question has a direct answer, because the answer is in the model and not in a supplement.

2.2. Modules couple through typed edges

A module declares its states, its parameters and its formalism: ODE or jump. It also declares how it touches the rest of the cell. INFERCELL has seven kinds of coupling edge (Table 2). Each kind says what crosses the edge and on which clock. Mass and currency edges act inside the ODE solve. Catalytic edges and deferred costs cross every second, at the handshake between the two blocks. Rate-constant edges

Table 1. Capabilities of whole-cell modelling frameworks. A tick means that the framework does this as part of its design. [Every cell to be checked against each framework's code and papers before submission.]

	Priors	Provenance in model	Inference on dynamics	Declared reductions
M. genitalium (Karr et al., 2012)	[?]	[?]	[?]	[?]
E. coli (Macklin et al., 2020)	[?]	[?]	[?]	[?]
Vivarium (Agmon et al., 2022)	[?]	[?]	[?]	[?]
JCVI-syn3A (Thornburg et al., 2022; 2026)	[?]	[?]	[?]	[?]
INFERCELL (this work)	✓	✓	✓	✓

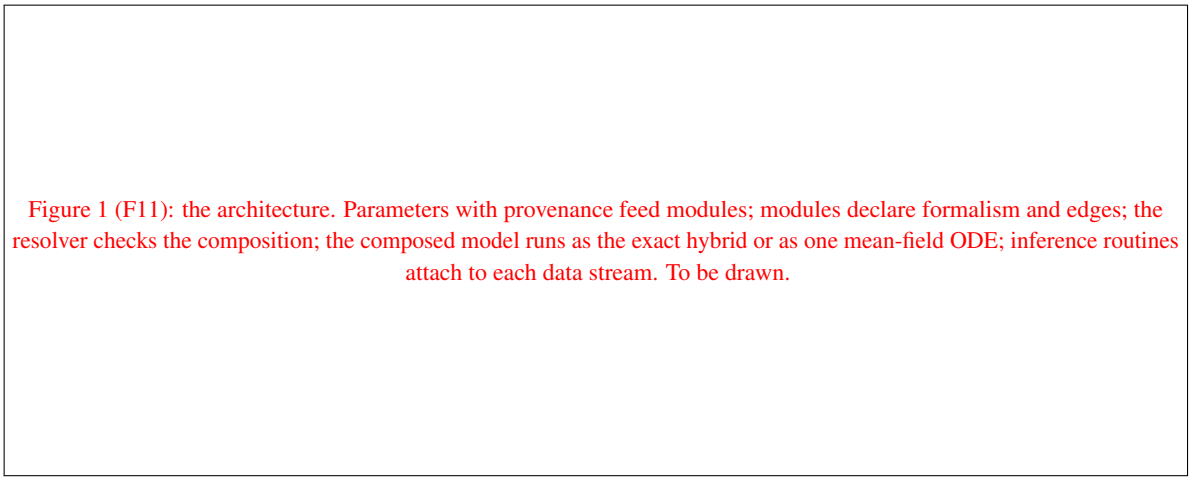


Figure 1. **The INFERCELL architecture.** [Caption to be written with the figure.]

Table 2. The seven edge kinds and their clocks.

Edge	What crosses	Clock
Mass	a shared continuous state	ODE solve
Currency	flux through a shared pool (ATP, GTP)	ODE solve
Catalytic	a protein count into a rate law	1 s
Deferred cost	jump-side cost, debited one step later	1 s
Rate constant	pools into stochastic rate constants	60 s
Volume	membrane proteins into cell geometry, and back	each step
Clamped	a dependence replaced by a constant	declared only

cross every minute, when the stochastic block rebuilds its rate constants from the metabolite pools. The set of kinds is closed. A module cannot invent a new kind of coupling without a change to the framework.

Before anything runs, a resolver reads all the edges and builds the coupling graph. It checks the composition for about twenty kinds of error. Examples are a state that two modules integrate, an edge that names a species no module owns, and a moiety that is consumed but never produced. In complete mode it refuses to build a model with an unowned state or a stranded moiety. A composition that runs is therefore a composition that the framework has checked.

2.3. Every reduction is declared

Any model of a cell leaves things out. In INFERCELL each departure from the published model is an object, not a comment. A reduction label has a category (one of 16, such as a clamp, a lumped step, a smoothed counter or an asserted prior) and a reason. The framework collects labels from three places: the edges, the parameter records and the modules. One call returns the full list for a composed model. This list is how we report the cost of each choice in Section 5.

2.4. One interface, two descriptions

Gene expression in INFERCELL has one interface and two descriptions.

The exact hybrid. The stochastic modules run as a jump process, and the metabolic modules run as ODEs. A driver steps the two blocks in turn, as the published model does. Every second it writes protein counts into the metabolic rate laws, steps the ODEs, debits the costs of the stochastic events, and steps the jump process. Every minute it rebuilds the stochastic rate constants from the pools. We did not wrap this in one solver object on purpose. A single jump problem

over an ODE problem would let the propensities follow the pools continuously. That is a different model from the published one. The driver can also record a stochastic path and replay it exactly. A replay matches the original run bit for bit at every handshake. The exact sampler (Appendix A.3) depends on this.

The mean-field swap. The gene-expression network of the model is first order. So the equations for the means of the transcript and protein counts close exactly, as linear ODEs. INFERCELL derives these equations from the same propensities and update rules that the jump modules use, so the two descriptions cannot drift apart. With the swap, the whole cell becomes one ODE problem with callbacks for the one-minute rebuild. Forward-mode automatic differentiation (Revels et al., 2016) passes through this problem and its callbacks. We test the gradients against central differences to a relative error of 10^{-6}.

What is and is not differentiable. The exact hybrid is not differentiable, because the published model is not. The published model clips a drained pool at zero and rounds pools to whole molecules. Both make the model a step function of its parameters. The exact sampler therefore uses no gradients. The mean-field description gives gradients for the parts of inference that need them. The cost of the swap is a declared reduction, and we measure it (Section 5.2).

2.5. Inference attaches to each data stream

A data stream declares its observation model. In this paper there are two streams: exact transcript counts per cell, and bulk metabolite measurements of the mean over many cells. INFERCELL has two inference routes.

- **The exact route.** A three-block Gibbs sampler conditions on the stochastic path. It updates the paths by conditional sequential Monte Carlo (Andrieu et al., 2010), the metabolic constants by slice sampling, and the gene-expression constants from exact one-dimensional conditionals (Appendix A.3).
- **The approximate route** (in development). The paths are integrated out. The transcripts keep an exact likelihood, and the bulk metabolites get a Gaussian likelihood centred on the mean-field model. The route is designed to use the gradients of the swap with the No-U-Turn Sampler (Hoffman & Gelman, 2014). [B5: built and tested; give the interface in one sentence.]

[B6, the dispatcher: a module declares its formalism and a stream declares its observation model; the dispatcher picks the route. Not built. If it is not built before submission, reword this section as a design principle and remove the claim.]

3. A reduced model of JCVI-syn3A

We test INFERCELL on a model reduced from the well-stirred JCVI-syn3A model of Thornburg et al. (2022). The reduction keeps the best-measured region of the cell and removes about 92% of it (Figure 2).

- **Metabolism:** 22 reactions in four ODE modules: glycolysis to lactate, sugar transport and lactate export, nucleotide recycling, and a lumped step for tRNA charging.
- **Gene expression:** 52 reactions in three jump modules: transcription, transcript decay and translation for 17 genes, and membrane insertion for the one membrane protein, the glucose transporter PtsG.
- **State:** 32 dynamic species and 5 held species, each named once in a registry with its initial value, prior width and source.

The two blocks couple in both directions, as in the published model. Enzyme counts set metabolic rates, and the costs of transcription and translation drain the nucleotide and charged-tRNA pools. In the other direction, the pools set the transcription and translation rate constants at each one-minute rebuild. Membrane protein counts set the cell's area and volume. No rate constant appears in both blocks. This property comes from the published model, not from our reduction.

The model makes no claim about JCVI-syn3A itself. With most of the cell removed, it cannot reproduce the doubling time, the metabolite levels or the proteome of the real organism. The code refuses to report a doubling time, so that no result can be compared with these quantities by mistake. The model's purpose is to give a real ODE–stochastic boundary from a published whole-cell model, at a size where an exact posterior is affordable.

M0, the reference system. An exact posterior on the full reduced model costs too much (Section 5.6). We therefore also use M0, the reduced model cut to two genes over 600 s. The two genes are GAPD, whose promoter is the best determined, and ptsG, the membrane transporter. All metabolism stays. The other enzyme counts are held at their measured copy numbers, and this hold is a declared reduction. M0 has six free parameters: the two promoter strengths, the transcript decay rate, the forward catalytic constants of enolase (ENO) and aldolase (FBA), and the bulk assay noise σ_{b}.

4. How information crosses the boundary

Three problems are often treated as one.

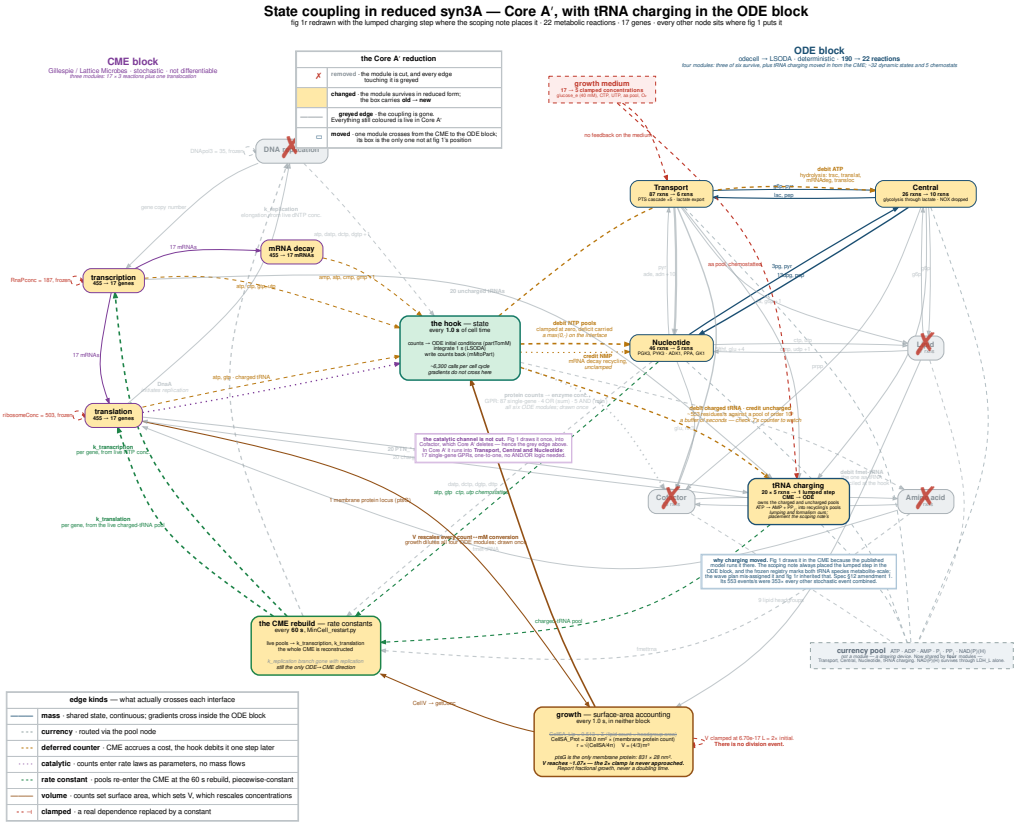


Figure 2. **The reduced JCVI-syn3A model.** Metabolic modules (ODE) and gene-expression modules (jump) and the edges between them. [Caption to be rewritten for the paper; the figure may need a cleaner version for print.]

- *Local inference:* what do a module's own data say about its own parameters?
- *Uncertainty propagation:* given the uncertain output of one module, how uncertain is the next module?
- *Cross-module inference:* do the data of one module inform the parameters of another, when the two share no parameter?

The third problem is the hard one, and it is the one this paper addresses.

4.1. Three graphs

A coupled model has three graphs, and they need not agree (Figure 3). The *coupling graph* says which module drives which state. The *likelihood graph* says which likelihood factors depend on which parameters once the stochastic paths are integrated out. The *information graph* says which data streams move which parameters. In the reduced JCVI-syn3A model the coupling graph is two-way, but the information graph is sparse and points mostly in one direction.

Figure 3: the coupling, likelihood and information graphs of the reduced model, side by side. To be drawn.

Figure 3. **Three graphs of one model.** [Caption.]

Write u for the trajectory at the interface: enzyme counts and drains in one direction, pools in the other. For a pair of modules $A \to B$ with data D_A and D_B, the forward message is $p(u \mid D_A)$. The backward message is a likelihood over the interface,

$$\lambda_B(u) = \int p(D_B \mid \theta_B, u)\, p(\theta_B)\, \mathrm{d}\theta_B. \qquad (1)$$

Table 3. Gain and information in each direction across the boundary. Information is the ratio of posterior to prior standard deviation from a linearised bound, at 200 cells and 10% assay noise; the bracket is the ratio that noise alone gives.

Direction	Gain	Information
Gene expression → pools	order one	ptsG from metabolites only: 0.10 (bracket 0.19)
Pools → rate constants	0.044–0.051 (transcription), 0.091 (translation)	ENO from transcripts only: 0.999

The joint posterior over u is proportional to $p(u \mid D_A)\,\lambda_B(u)$. A *statistical cut* drops λ_B (Plummer, 2015). It changes the target of inference, and data from B can no longer inform A. A *dynamical reduction* instead removes a weak edge from the simulator and keeps full Bayesian inference. For the question we ask, only the second kind keeps the information we want: metabolite data that inform a gene-expression parameter.

4.2. Information follows gain

The strength of each coupling direction predicts where information flows (Table 3). From gene expression to the pools the gain is of order one. The pools turn over in seconds to minutes, and the drains from gene expression are a large part of that turnover. From the pools back to the rate constants the gain is small: an elasticity of 0.044–0.051 for transcription and 0.091 for translation. The gain is small because the published rate laws are saturated. The half-saturation constant of RNA polymerase for its nucleotides is 0.1 mM, while ATP is near 3.65 mM.

So metabolite data can inform the ptsG promoter, a gene-expression parameter that they never measure. Transcript data carry almost no information about the enolase constant, a metabolic parameter. We report the second result as a measured null, not as a failure. [FBA under transcripts only is open; K2 at 1,000 cells decides it (16b.5).]

4.3. Why no parameter is shared

The standard modular methods pass messages through shared parameters: cuts and semi-modular inference (Plummer, 2015; Carmona & Nicholls, 2020), and Markov melding (Goudie et al., 2019). In the reduced model no parameter is shared, so these methods have nothing to pass while we condition on the stochastic path. The exact sampler does what is left: it conditions on the path, and it updates the path with all the data. In this sense the exact sampler is modular inference with nothing dropped. When the paths are integrated out, a gene-expression parameter appears in both the transcript factor and the metabolite factor. The approximate route uses this structure.

5. Results

All results use synthetic data. The data come from the full two-way model at known parameter values, so the truth is known. The test of an inference method is then calibration: over many datasets, a 90% credible interval must contain the truth about 90% of the time (Talts et al., 2018).

5.1. The exact sampler is calibrated on M0

Convergence. We ran 4 chains of 1,650 sweeps on one M0 dataset of 50 cells. The rank-normalised $\hat{R}$ (Vehtari et al., 2021) is at most 1.007, and the bulk effective sample size is at least 1,081 for every parameter. This reference run cost 6,810 CPU-hours.

Calibration. We drew 50 parameter sets from the prior, simulated one dataset from each, and fitted each dataset with one chain. For each parameter we compared the empirical CDF of the ranks of the truth with a simultaneous band (Säilynoja et al., 2022), at a joint level of 95% over the six parameters. All six parameters stay inside their bands (Figure 4a). The largest deviation is 0.164, for the transcript decay rate, where the band allows 0.244. We also read coverage off the same ranks, for central credible intervals at five levels from 50% to 99%. In 29 of 30 parameter–level pairs the coverage is inside the binomial 95% band. One or two misses over 30 pairs are expected by chance. The one miss is the decay rate at 99%, where $L = 100$ draws per chain give the coarsest ranks.

A production-length chain matches the reference. One chain of the length we used in the calibration runs is not distinguishable from the reference on any marginal (Figure 4b). The means differ by at most 0.12 reference standard deviations, and the ratio of standard deviations is between 0.91 and 1.19. This chain cost 507.5 CPU-hours, about 13 times less than the reference.

Limits. The calibration result holds for M0 only. In 23 of the 50 replicates the smallest bulk effective sample size is below 100. The production chain resolves the FBA constant from its matched null by a small margin, 0.020 standard deviations.

5.2. The cost of the formalism swap is declared and measured

The mean-field swap replaces the stochastic gene expression with its mean. The means of transcripts and proteins are exact, but metabolism is nonlinear. A metabolism fed the mean drain differs from the mean over cells of metabolisms fed their own drains. The difference is a Jensen gap.

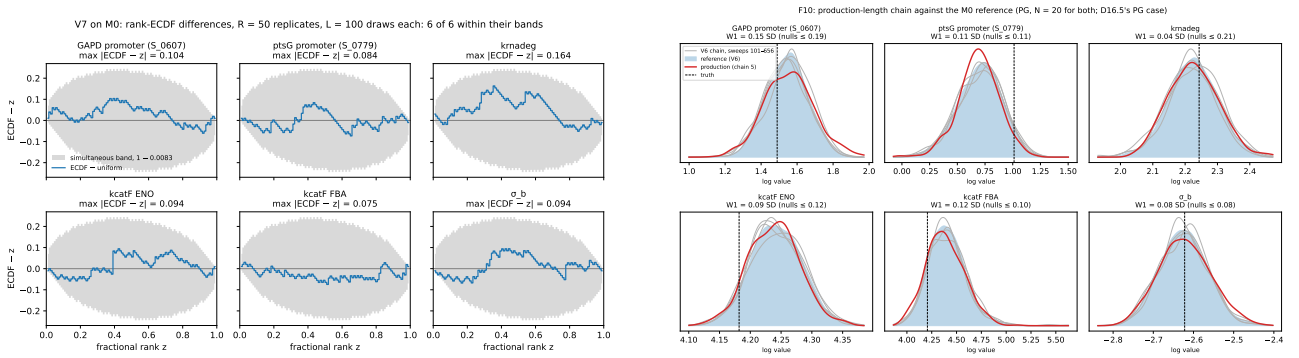


Figure 4. **The exact sampler on M0.** (a) Rank-ECDF of the truth over 50 prior-drawn datasets, against simultaneous 95% bands. (b) Marginal posteriors from a production-length chain against the reference. [Restyle both panels to one figure with shared fonts; label panels.]

The cause. The gap is largest for GTP, where the mean field sits 10–16% below the population mean. We traced it to cell-to-cell variation in the drains. When every cell receives the mean field's drains, the mean over cells equals the mean field to three significant figures. We ruled out the enzyme counts, the per-cell feedback and the clip at zero as causes of the GTP gap. A likely mechanism, which we have not tested, is that only saturable enzymes regenerate GTP. Then cells that spend more GTP lose more of it than cells that spend less gain back, and the population mean sits above GTP at the mean spend.

The correction. We treat the gap as a declared reduction and correct it with an offset δ, the log difference between the mean of a reference ensemble and the mean field at one reference parameter set. Our reference is the most detailed composition we have, the exact hybrid, and not the truth about the cell. We report the gap with a goodness check and with the parameter bias it implies, and we do not use it as a stop.

- Without δ, the gap would move the transcript decay rate by 2.4–3.1 posterior standard deviations.
- With δ, the remaining bias is within about one standard deviation.
- The goodness check as first written fails a description known to be correct in about half of all cases: a correct description passes only 36–56% of the time. We therefore compare the check with its own null distribution. With δ, the mean field falls inside the null's 95% range at 26 of 28 targets, against 5 of 28 without δ.

The mean-field model is the limit of the hybrid. The hybrid exchanges state every second. As this step shrinks, the hybrid's mean converges to the one-ODE mean-field model. At the zero-step limit the two agree to 0.0122 in log concentration at all five test parameter sets. One mean-field solve takes 1.1 s on M0 and 9–12 s on the full reduced model.

[AV6 (B7) decides whether the remaining gap moves the posterior. Add one sentence with the answer.]

5.3. The approximate posterior agrees with the exact one on M0

[B7 / AV6, not run. Refit the 50 V7 datasets with the approximate route. Report: overlaid marginals and the ptsG–ENO contour against the exact posterior; rank-ECDF of the approximate route; cost per posterior against 507.5 CPU-hours. Gate: M0 carries a metabolite → ptsG signal (linearised ratio 0.47 against a noise bracket of 0.68; met).]

5.4. Inference on the full reduced model

[B8 / AV7, not run; needs the cap reopened (O1). Calibration over ≥ 200 injections on the full reduced model. Posterior widths against the linearised information bound for the six D11 targets. The F6 table: transcripts only, metabolites only, both. Target from the bound: ptsG under metabolites only at 0.10 of its prior SD (bracket 0.19); ENO at 0.13 (bracket 0.75).]

5.5. The uncertainty budget of a prediction

[Not built (outline §6). Draw the inferred parameters from the posterior and all others from their priors; run the coupled model; split the variance of an unobserved quantity (fractional growth, lactate export, or how often GTP empties) into: inferred parameters, parameters left at their priors, and intrinsic noise. Report the split with asserted prior widths halved and doubled. Target sentence: "x% of the uncertainty in this prediction comes from parameters that no measurement supports." About 50 CPU-h on M0.]

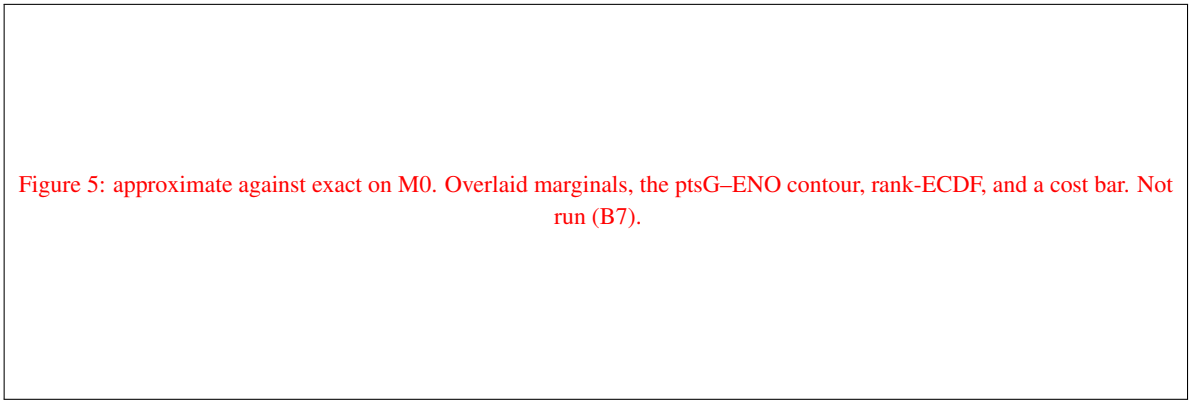


Figure 5. **Approximate against exact.** [Caption.]

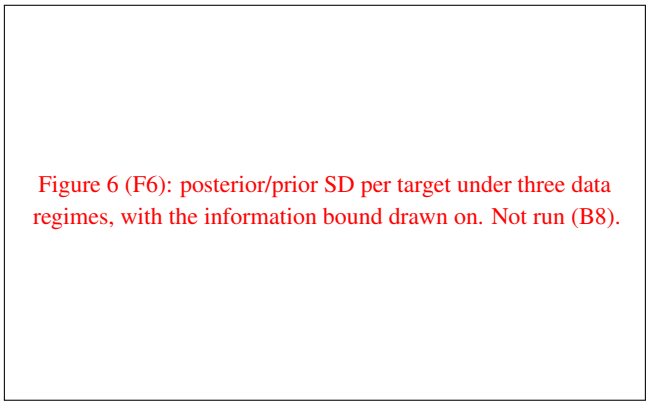


Figure 6. **What each data stream informs.** [Caption.]

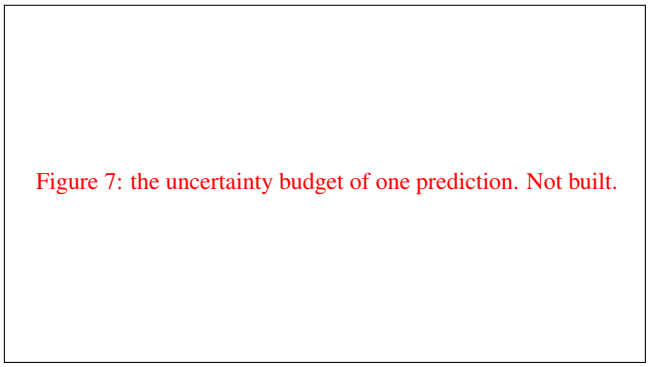


Figure 7. **Where the uncertainty of a prediction comes from.** [Caption.]

Table 4. CPU-hours per posterior.

Route	M0	Reduced model
Exact, reference	6,810	—
Exact, production	507.5	∼21,000 (projected)
Approximate	[B7]	[B8]

5.6. Cost

Table 4 gives the cost of one posterior by each route. An exact posterior on the full reduced model is projected at about 21,000 CPU-hours, above our bound of 1,000 CPU-hours per posterior. Exact inference therefore stops at M0. The cost is the number of simulations, not the time of one. One cycle of the full reduced model takes 32.2 s, and a fast implementation of the full JCVI-syn3A model is the subject of a companion paper (Kimpson, 2026).

6. Discussion

[Opening paragraph: restate the main result once B7 and B8 are in.]

What scales and what does not. Exact inference across the boundary works, and it is calibrated, but it stops at M0. The cost of an exact posterior grows with the number of genes and the length of the record, and the full reduced model is already above our budget. The approximate route removes the stochastic paths from the sampler. Two quantities limit it. The first is the Jensen gap, which we declare and correct (Section 5.2). The second is R, the variance of the bulk log-likelihood over the paths that the transcripts allow. R is about 64 on M0 and 70 on the full reduced model at 10% assay noise. So the metabolites constrain the paths together, and a single draw of paths per parameter value is not enough. The Gaussian bulk factor handles this regime. [Confirm once B7 reports.]

The observation model sets the interface. The three graphs of Section 4 give a general rule. Single-cell data inform the parameters of their own module. Information between modules travels on bulk observables, and it follows the gain of the coupling. A modeller can compute the gains before any inference, and in this work the gains predicted the information in each direction before any posterior existed.

Reductions as approximations to inference. A dynamical reduction changes the simulator and keeps full Bayesian inference. A statistical cut keeps the simulator and changes the target. We prefer the first, because its cost can be measured against a more detailed composition. The formalism swap is such a reduction. We do not claim that the more detailed composition is the truth about the cell. We claim that each choice of description is declared, and that its cost is measured.

6.1. Limitations

- *Synthetic data only.* The data come from the model itself. Real assays need observation models for unknown cell numbers in bulk metabolomics and for capture efficiency in single-cell RNA counts.
- *One organism, one reduction.* We demonstrate the framework on one reduced model. [O3: build the generic registry, or narrow the framework claim to "demonstrated on one organism".]
- *Asserted priors.* Many priors have no width in their source, so we set the width. The uncertainty budget (Section 5.5) reports how much this choice matters. [Once built.]
- *Exact calibration at small scale only.* Coverage of the exact sampler at the full reduced model is out of reach at about 21,000 CPU-hours per posterior.
- *Weak reverse coupling.* The approximate route relies on the small elasticities of the rebuild (0.044–0.091). A perturbation that drains the nucleotide pools toward the half-saturation constant would remove this basis.

6.2. Future work

[One short paragraph, in this order: (1) the method's own limits: condition the bulk factor on the observed transcripts; path sampling only for pools near the one-particle floor; strong-feedback regimes. (2) The full well-stirred JCVI-syn3A model through the syn3a-ensemble port, then real data. (3) Comparing model structure and formalisms, which the swap makes possible, by predictive scores rather than Bayes factors. (4) Goal-directed inference and experimental design: which parts of a coupled simulator must be kept, and which latent variables inferred, to keep the information a prediction needs?]

Software and data availability

INFERCELL is open source at `https://github.com/MACSYS-CoE/InferCell`, with documentation at `https://macsys-coe.github.io/InferCell/`. The test suite has 28,709 assertions. [Release tag and archived DOI (Zenodo) for the version used in this paper.] [Archive of the posterior draws and result files, with the scripts and commits that produced each number.]

Acknowledgements

[Funding, computing resources (OzSTAR), collaborators.]

References

Agmon, E., Spangler, R. K., Skalnik, C. J., Poole, W., Peirce, S. M., Morrison, J. H., and Covert, M. W. Vivarium: An interface and engine for integrative multiscale modeling in computational biology. *Bioinformatics*, 38(7):1972–1979, 2022. doi: 10.1093/bioinformatics/btac049.

Andrieu, C., Doucet, A., and Holenstein, R. Particle Markov chain Monte Carlo methods. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 72(3): 269–342, 2010. doi: 10.1111/j.1467-9868.2009.00736.x.

Babtie, A. C. and Stumpf, M. P. H. How to deal with parameters for whole-cell modelling. *Journal of The Royal Society Interface*, 14(133):20170237, 2017. doi: 10.1098/rsif.2017.0237.

Bezanson, J., Edelman, A., Karpinski, S., and Shah, V. B. Julia: A fresh approach to numerical computing. *SIAM Review*, 59(1):65–98, 2017. doi: 10.1137/141000671.

Carmona, C. and Nicholls, G. Semi-modular inference: Enhanced learning in multi-modular models by tempering the influence of components. In *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics*, volume 108 of *Proceedings of Machine Learning Research*, pp. 4226–4235. PMLR, 2020.

Gillespie, D. T. Exact stochastic simulation of coupled chemical reactions. *The Journal of Physical Chemistry*, 81(25):2340–2361, 1977. doi: 10.1021/j100540a008.

Goudie, R. J. B., Presanis, A. M., Lunn, D., De Angelis, D., and Wernisch, L. Joining and splitting models with Markov melding. *Bayesian Analysis*, 14(1):81–109, 2019. doi: 10.1214/18-BA1104.

Hobolth, A. and Stone, E. A. Simulation from endpoint-conditioned, continuous-time Markov chains on a finite state space, with applications to molecular evolution. *The Annals of Applied Statistics*, 3(3):1204–1231, 2009. doi: 10.1214/09-AOAS247.

Hoffman, M. D. and Gelman, A. The No-U-Turn sampler: Adaptively setting path lengths in Hamiltonian Monte Carlo. *Journal of Machine Learning Research*, 15(47):1593–1623, 2014. URL `http://jmlr.org/papers/v15/hoffman14a.html`.

Jacob, P. E., Murray, L. M., Holmes, C. C., and Robert, C. P. Better together? Statistical learning in models made of modules, 2017. arXiv:1708.08719.

Karr, J. R., Sanghvi, J. C., Macklin, D. N., Gutschow, M. V., Jacobs, J. M., Bolival, B., Assad-Garcia, N., Glass, J. I., and Covert, M. W. A whole-cell computational model predicts phenotype from genotype. *Cell*, 150(2):389–401, 2012. doi: 10.1016/j.cell.2012.05.044.

Kimpson, T. A whole minimal cell in a minute: A validated, fast implementation of the hybrid JCVI-syn3A model, 2026. In preparation.

Macklin, D. N., Ahn-Horst, T. A., Choi, H., Ruggero, N. A., Carrera, J., Mason, J. C., Sun, G., Agmon, E., DeFelice, M. M., Maayan, I., Lane, K., Spangler, R. K., Gillies, T. E., Paull, M. L., Akhter, S., Bray, S. R., Weaver, D. S., Keseler, I. M., Karp, P. D., Morrison, J. H., and Covert, M. W. Simultaneous cross-evaluation of heterogeneous E. coli datasets via mechanistic simulation. *Science*, 369 (6502):eaav3751, 2020. doi: 10.1126/science.aav3751.

Plummer, M. Cuts in Bayesian graphical models. *Statistics and Computing*, 25(1):37–43, 2015. doi: 10.1007/s11222-014-9503-z.

Rackauckas, C. and Nie, Q. DifferentialEquations.jl – a performant and feature-rich ecosystem for solving differential equations in Julia. *Journal of Open Research Software*, 5(1):15, 2017. doi: 10.5334/jors.151.

Revels, J., Lubin, M., and Papamarkou, T. Forward-mode automatic differentiation in Julia, 2016. arXiv:1607.07892.

Säilynoja, T., Bürkner, P.-C., and Vehtari, A. Graphical test for discrete uniformity and its applications in goodness-of-fit evaluation and multiple sample comparison. *Statistics and Computing*, 32(2):32, 2022. doi: 10.1007/s11222-022-10090-6.

Talts, S., Betancourt, M., Simpson, D., Vehtari, A., and Gelman, A. Validating Bayesian inference algorithms with simulation-based calibration, 2018. arXiv:1804.06788.

Thornburg, Z. R., Bianchi, D. M., Brier, T. A., Gilbert, B. R., Earnest, E. E., Melo, M. C. R., Safronova, N., Sáenz, J. P., Cook, A. T., Wise, K. S., Hutchison, C. A., Smith, H. O., Glass, J. I., and Luthey-Schulten, Z. Fundamental behaviors emerge from simulations of a living minimal cell. *Cell*, 185(2):345–360.e28, 2022. doi: 10.1016/j.cell.2021.12.025.

Thornburg, Z. R., Maytin, A., Kwon, J., Brier, T. A., Gilbert, B. R., Fu, E., Gao, Y.-L., Quenneville, J., Wu, T., Li, H., Long, T., Pezeshkian, W., Sun, L., Bittencourt, D. M. d. C., Glass, J. I., Mehta, A. P., Ha, T., and Luthey-Schulten, Z. Bringing the genetically minimal cell to life on a computer in 4D. *Cell*, 189(9):2582–2597.e27, 2026. doi: 10.1016/j.cell.2026.02.009.

Vehtari, A., Gelman, A., Simpson, D., Carpenter, B., and Bürkner, P.-C. Rank-normalization, folding, and localization: An improved $\widehat{R}$ for assessing convergence of MCMC (with discussion). *Bayesian Analysis*, 16(2):667–718, 2021. doi: 10.1214/20-BA1221.

A. Methods

[Venue decides the form: STAR Methods (Cell Systems) or a Methods section after the Discussion (PLOS CB). Content is the same.]

A.1. The hybrid driver

The driver keeps two blocks with separate state: metabolite pools in mM as floating point values, and molecule counts as integers. One handshake step does the following, in order:

1. update growth and cell geometry from the membrane protein counts;
2. write the protein counts into the metabolic rate laws (catalytic edges);
3. step the ODE block by 1 s (Rodas5P, absolute tolerance 10^{-10}, relative tolerance 10^{-8});
4. debit the pools by the costs of the stochastic events of the previous step (deferred-cost edges), clipped at zero as in the published model;
5. every 60 s, rebuild the stochastic rate constants from the pools;
6. step the jump block by 1 s with Gillespie's direct method (Gillespie, 1977).

On the jump clock the rebuilt constants therefore take effect at $60m - 1$ s. Pools are written back as molecule counts under a declared rounding policy. A recorded path can be replayed: the replay applies the recorded firings instead of drawing new ones, and it returns the log density of the path and its sufficient statistics (event counts and exposures).

A.2. Observation model

Each synthetic dataset has C cells (50 on M0), observed every 60 s.

- **Transcripts:** the exact count of each gene's transcripts in each cell at each save.
- **Bulk metabolites:** one measurement per pool per save of the mean over the C cells, $\bar{x}$, with lognormal noise: $\log z \sim \mathcal{N}(\log \bar{x}, \sigma_{\mathrm{b}}^2)$. The assay noise σ_{b} is a free parameter.

We use bulk metabolites because a per-cell metabolite panel cannot be measured. [State the panel (M0_PANEL) and the prior on σ_{b}.]

A.3. The exact sampler

The sampler is a three-block Gibbs scan with the stochastic path as a latent variable.

1. **Paths.** Conditional sequential Monte Carlo over 60 s windows (Andrieu et al., 2010), with particle Gibbs at $N = 20$ particles. Each particle is a snapshot of the driver. Transcripts are proposed as exact endpoint-conditioned birth–death bridges (Hobolth & Stone, 2009), so every proposal agrees with the observed transcript counts. The other reactions are then simulated given the transcripts. The weight is the bulk metabolite likelihood times each gene's transition probability. Cells are updated in turn, because the bulk likelihood couples them.
2. **Metabolic constants and σ_{b}.** A univariate slice step on each log parameter, given the paths. Reverse constants follow from the forward ones and the equilibrium constants.
3. **Gene-expression constants.** Given the path, each promoter strength and the decay rate have an exact one-dimensional conditional of the form $nu - e^u A + \log p(u)$, where n is the event count and A the exposure. We sample it by a slice step.

The sampler is written directly, not in a probabilistic programming language. The path update needs ancestor sampling and exact snapshots of the driver, and a general framework could not provide them without sharing solver state between particles. Runs can be checkpointed and resumed bit for bit, and the threaded sweep is bit for bit equal to the serial one.

A.4. The transcript factor

When the paths are integrated out, each gene's transcripts form a birth–death chain with a piecewise-constant birth rate, observed exactly every 60 s. The likelihood of one cell's record is a product of transition probabilities. We compute them by uniformisation with tridiagonal products, truncated where the remaining mass is below 10^{-12}. The cap on the count is fixed per dataset, and the number of terms comes from the primal value, so the factor is differentiable in the rates.

A.5. The mean-field route

The swap replaces the three jump modules with one mean-field module. Its equations are generated from the jump modules' own propensities and update rules. The composed model is one ODE problem. Four edges change form, and each change is a declared reduction:

- catalytic edges read the mean protein counts;
- drains are continuous and not clipped, and the solve stops if a pool becomes negative;
- the rebuild is a discrete callback at $60m - 1$ s and at $t = 0$;
- growth is continuous.

The bulk factor is $\mathcal{N}\big(\log z;\ \log\max(\bar{x}_{\mathrm{MF}}(\theta), 1) + \delta,\ \sigma_{\mathrm{b}}^2 I + S\big)$, with δ and S from an ensemble of the exact hybrid at a reference parameter value, and the floor of one molecule. [B5: state how S is estimated (delta method over 800 cells) and the sampler settings once built.]

A.6. Simulation-based calibration

For each of $R = 50$ replicates we drew a parameter set from the prior, simulated a dataset, and ran one chain of 656 sweeps from a prior draw. We dropped 100 sweeps and thinned the rest to $L = 100$ draws. The rank of the truth among the draws is uniform if the sampler is calibrated (Talts et al., 2018). We tested each parameter's rank-ECDF against a simultaneous band at $1 - 0.05/6$ (Säilynoja et al., 2022), so the test as a whole is at the 95% level. The pass rule was fixed before any rank was computed.

A.7. Compute

All runs used the OzSTAR cluster. [Hardware, total CPU-hours, and a table of every job behind each number, from the result files.]